Device Recognition & Control — Team Guide

Homework — Pr Belkhir Abdelkader · remise 30 octobre 2026

Oct 16, 2026 · @project

The assignment

Two deliverables, one report, due 30 October 2026.

1. **Reconnaissance** — a program that detects every peripheral on the machine and prints the characteristics of each one.
2. **Prise de contrôle du clavier** — take control of the keyboard. The professor fixed the device: it is the keyboard, not one of our choosing.

Constraints from the sheet: the work runs on Linux **or** Windows, the report is **4 pages maximum**, and the group is **5 students**. We do it on Windows.

The real constraint is time. Today is 16 October — the deadline is **14 days away**, and a Windows driver eats days (signing, the device stack, a dead keyboard on reboot). The plan below front-loads the risky parts into this week and keeps code out of the report, so the 4 pages stay a summary of serious work.

Our approach — the hard path

On Windows you can't pull the keyboard away from its driver the way you can on Linux, so instead we insert our own code into the keyboard's input path — in user space first, then as a driver inside the Windows input stack.

The easy version of this homework toggles the Caps Lock LED with one `keybd_event` call and calls it "control". We go deeper, because the keyboard on Windows travels through a stack (HID → kbdhid → kbdclass → Win32) and a cybersecurity course is the right place to show how to sit inside it. Three layers, each working before the next:

Layer	What we build	Runs in
1	Enumerator that walks the PnP device tree, finds the keyboard, and prints its hardware ID (VID/PID), device instance path and driver	user space
2	Low-level keyboard hook that intercepts every keystroke before applications see it — log, block or remap — plus LED control sent to the class driver	user space

Layer	What we build	Runs in
3	Our own kernel-mode keyboard filter driver (KMDF) that sits in the keyboard device stack and reads every scan code	kernel space

Layer 3 is the headline, and it runs as two tracks. We ship the first for certain:

- **Low-level hook module** (guaranteed) — `SetWindowsHookEx(WH_KEYBOARD_LL)` installs a system-wide hook; our callback logs every key and can swallow or remap it. User space, no driver signing, and it cannot kill the keyboard. The safe, on-topic core of Part 2.
- **KMDF keyboard filter driver** (stretch) — our own filter modelled on Microsoft's `kbfiltr` sample, attached to the kbdclass stack, reading `KEYBOARD_INPUT_DATA` scan codes in the kernel. Harder, and a bad one can leave the VM with no keyboard — so it lives in a VM with test-signing on.

Safety rule. All driver work happens in a virtual machine with a snapshot before each install, and test-signing mode turned on (`bcdedit /set testsigning on`). A broken keyboard filter driver can dead-key the machine on the next boot; in a VM that costs a snapshot rollback, on a laptop it costs the afternoon. Keep the Windows recovery option "Disable driver signature enforcement" in mind as the way back in.

Part 1 — Enumeration

We walk Windows' own device database instead of parsing the output of someone else's tool.

Every device Windows knows about is a Plug and Play node reachable through the SetupAPI device-information functions (`SetupDiGetClassDevs`, `SetupDiEnumDeviceInfo`, `SetupDiGetDeviceRegistryProperty`) and the Configuration Manager (`CM_Get_Parent`, `CM_Get_Child` for the tree). The high-level view of the same data is the WMI class `Win32_PnPEntity`, which PowerShell's `Get-PnpDevice` wraps. Reading it ourselves is the difference between using a tool and understanding one.

For each device we print: friendly name and device instance path, device class and its GUID, hardware IDs (vendor and product for USB), manufacturer, the bound driver or service, and PnP status. For the keyboard we also resolve its parent chain.

Two implementations, written against the same output format so we can compare them in the report:

- **C + SetupAPI / CfgMgr32** — the serious one, talking straight to the PnP manager. This is what Windows' own device tools are built on.

- **PowerShell `Get-PnpDevice` + Python (`wmi` module)** — the same walk in a fraction of the code, used to cross-check the C output and to generate the tables for the report.

The interesting section of the report is not the listing itself. It is the device **tree**, traced down to our keyboard: the HID keyboard hangs off a USB input device, off a USB hub, off the xHCI host controller on the PCI root — one hierarchy from CPU to key, built from the Configuration Manager's parent/child links. And this step is not just for show: the keyboard's hardware ID and device instance path are exactly what Part 2 needs to attach to it, so Part 1 ends by printing them.

If one member wants to cover Linux for a comparison paragraph, the equivalent is libudev (`udev_enumerate_scan_devices`) reading the `/sys` device tree — worth one line in the report, no more.

Part 2 — Taking control

We put our own code into the keyboard's input path and drive the device — first from user space, then from inside the kernel's keyboard stack.

Step A — intercepting keystrokes from user space

On Windows you don't detach the keyboard driver; you hook the input path. The sequence:

1. `SetWindowsHookEx(WH_KEYBOARD_LL, ...)` installs a system-wide low-level keyboard hook. No driver, no signing.
2. Our `LowLevelKeyboardProc` callback fires on every key down/up before any application sees it. From the `KBDLLHOOKSTRUCT` we read the virtual-key code and scan code, map them to characters, and print them — capturing keystrokes ourselves is control.
3. Return non-zero instead of calling `CallNextHookEx` to **swallow** a key (blocking, say, the Windows key) or change it to remap. A message loop (`GetMessage`) must run or the hook silently does nothing.
4. Read the physical device with the Raw Input API (`RegisterRawInputDevices` on the keyboard usage page) so we can tell which keyboard a key came from.
5. Toggle the Caps/Num/Scroll LEDs: the simple way is `SendInput` on the lock keys; the hard way is a `DeviceIoControl` with `IOCTL_KEYBOARD_SET_INDICATORS` to the keyboard class driver.

This is visible, safe, and runs on the real machine — the hook cannot brick the keyboard.

Step B — our own kernel-mode driver

The real deliverable. Two tracks; we ship the first and attempt the second.

Track 1 — the hook, polished (guaranteed). The Step A low-level hook built into a clean tool: logs every keystroke to a console and a file, blocks a chosen key, and drives the LEDs. User-mode, so it always demos. The floor we will not miss.

Track 2 — KMDF keyboard filter driver (stretch). Our own filter driver modelled on Microsoft's `kbfiltr` sample: it attaches to the kbdclass device stack, and its read-completion callback sees every key's `KEYBOARD_INPUT_DATA` (scan code, make/break flag) in the kernel before the system does. It can log, drop or remap — exactly what Sysinternals' Ctrl2Cap does to turn Caps Lock into Ctrl. Built with Visual Studio + the WDK, installed with its INF, logged with `DbgPrint` to DebugView.

Where it will hurt, so nobody loses two days: 64-bit Windows refuses unsigned drivers (Driver Signature Enforcement), so turn on test-signing (`bcdedit /set testsigning on`) and sign with a test certificate — VM only; the WDK version must match the Visual Studio version; a broken keyboard filter can dead-key the machine on boot, so snapshot before every install and know the "Disable driver signature enforcement" boot option; and the low-level hook needs a running message loop or it does nothing.

GitHub repositories

Clone the driver-samples repo first and build `input/kbfiltr`. The rest is reference.

Repository	What's in it	Why we need it
microsoft/Windows-driver-samples — kbfiltr	Microsoft's official KMDF keyboard filter driver sample — attaches to kbdclass, callbacks on every scan code, with an INF	The exact template for Track 2. Start here.
microsoft/Windows-driver-samples	The whole driver-samples tree (build with VS + WDK)	Parent repo; `input/moufiltr` is the mouse analogue if we want to compare.
oblitum/Interception	A ready, signed keyboard/mouse interception driver plus a simple user-space library	Fallback and reference — a working filter driver we can read and run if ours fights us.

Repository	What's in it	Why we need it
Ctrl2Cap (Sysinternals)	The classic minimal keyboard filter driver — remaps Caps Lock to Ctrl	The smallest real filter driver to read before writing ours.

The enumerator (Part 1) and the low-level hook (Step A) are short enough that we write them from the Microsoft docs rather than clone them — see the next section.

One note: the driver-samples repo is large. Don't build all of it — open the `input/kbfiltr` solution alone. And install a WDK whose version matches your Visual Studio.

Video resources

Windows kernel-driver video is scarcer than Linux's — there is no single canonical playlist like the Linux one. The fastest path is one written walkthrough plus the kbfiltr source; use video to fill gaps.

Resource	Link	Use it for
Write your first driver (Microsoft, written)	learn.microsoft.com	The canonical first build → deploy → test-sign loop
Create a Windows kernel driver in 1 hour (walkthrough)	it4sec.substack.com	A fast, modern end-to-end driver build
Kbfiltr sample page (Microsoft)	learn.microsoft.com	Build notes for the driver we base Track 2 on

For actual video, search YouTube for *Pavel Yosifovich — Windows Kernel Programming* (his conference talks) and *kbfiltr walkthrough*. Whoever owns Track 2 watches one and reports back; the rest of us read the kbfiltr source, which teaches more than any video.

Documentation and books

Cite Microsoft Learn and the kbfiltr sample — both free. For the keyboard specifically, a few references matter.

Write your first driver — Microsoft's getting-started path: install Visual Studio + the WDK, build, deploy to the target, test-sign. Do this first.

Kbfiltr sample and the **KMDF sample index** — the filter driver Track 2 is based on, and the wider KMDF sample list.

SetupAPI device-information functions — `SetupDiGetClassDevs` is the entry point for Part 1: learn.microsoft.com. Its siblings `SetupDiEnumDeviceInfo` and `SetupDiGetDeviceRegistryProperty` are linked from there.

The input path — `SetWindowsHookEx` with `WH_KEYBOARD_LL` and the `LowLevelKeyboardProc` callback for Step A, and the **Raw Input** guide for reading the physical device.

Keyboard class-driver IOCTLs — `ntddkbd.h`, including `IOCTL_KEYBOARD_SET_INDICATORS` for the LEDs: learn.microsoft.com.

Book: Pavel Yosifovich, *Windows Kernel Programming* — the standing reference for the driver track; the WDM-era classic is Walter Oney's *Programming the Microsoft Windows Driver Model*.

Two more: **DebugView** (Sysinternals) to see `DbgPrint` output from the driver, and **OSR Online** (community.osr.com), the Windows-driver community where kbfiltr questions get answered.

Who does what

Five seats, each owning something that can be demoed alone. The two kernel-adjacent seats run independent tracks, so a stuck filter driver never sinks the demo.

#	Seat	Owns
1	Enumeration (C)	SetupAPI / CfgMgr32 enumerator, the device tree, printing the keyboard's hardware ID and instance path
2	Enumeration (scripts)	PowerShell `Get-PnpDevice` + Python `wmi` cross-check; the tables for the report
3	User-space control	Low-level hook: intercept, block a key, Raw Input, LED toggle
4	Kernel driver	The stretch `kbfiltr`-based KMDF filter; test-signing; VM. Pairs with seat 3.
5	Lab + report	VM with snapshots, Visual Studio + WDK, test-signing setup, DebugView screenshots, the 4 pages

Seats 3 and 4 are the pair to watch. If the filter driver defeats us in the time, seat 4 folds into helping seat 3 and the report documents the attempt — the hook is the deliverable that must ship. Seat 5 is not the easy seat: that person builds the environment everyone

depends on and writes the document that gets graded.

Timeline — 14 days

1. **16–19 Oct (this week)** — VM with a snapshot, Visual Studio + WDK installed, test-signing on. Everyone builds and loads a hello-world KMDF driver. Seat 1 identifies the keyboard and notes its hardware ID. Seat 3 gets a hook logging keys.
2. **20–26 Oct** — Part 1 finished in C + PowerShell. Seat 3 done: intercept + block + LED. Seat 4 has `kbfiltr` building and attached to the keyboard stack, logging scan codes in DebugView — or the honest attempt documented. Everything tested on a clean snapshot. Report drafting starts.
3. **27–30 Oct** — freeze the code on **27 Oct**. Three days for the demo rehearsal (type, show DebugView capturing scan codes, toggle the LEDs, block a key) and the final read-through. **Submit 30 Oct.**

The freeze date is the one that matters. A driver that works on the 29th and dies during the demo is worth less than one that stopped changing on the 27th.

The report — 4 pages

Four pages is roughly 1,600 words. Budget them before writing, or page 4 gets cut mid-sentence.

Page	Content
1	Problem statement; the Windows device model (PnP, SetupAPI, the keyboard stack HID → kbdhid → kbdclass) in half a page; why the filter-driver route
2	Part 1: the SetupAPI enumerator, one screenshot of the device tree with the keyboard located, a short table of what we read per class
3	Part 2: the low-level hook intercepting and blocking keys + LED toggle, then the kbfiltr filter driver. The key screenshot: DebugView showing our driver logging real scan codes.
4	Results; Track 2 (filter driver) done or honestly attempted; what failed and why; references

What keeps it under four pages: **no full code listings**. Put the repo link in the header and name file names in the text. Code in a report is wasted space — the reader either trusts it or opens the repo.

What earns marks: the **DebugView** screenshot of our own filter driver logging scan codes as someone types. That single image proves our code sits in the keyboard stack — the whole point of taking control — and almost nobody else will have it.

Section 4 should say honestly what did not work. A report admitting the filter driver defeated us in 14 days, with a clear why (Driver Signature Enforcement, the device stack), reads better than a vague claim that everything worked.